Concurrency

Description

Why Concurrency?

- ***Multiple Things at Once*** **(MTAO):** Modern operating systems must manage many activities simultaneously. Concurrency allows the OS to handle:
  - Multiple processes and threads concurrently.
  - Asynchronous events such as interrupts and background tasks.
  - System maintenance tasks running in parallel.
- **Server Environments:** For instance, servers must handle many simultaneous connections.
- **Parallel Programs:** Achieving higher performance by dividing work among multiple execution units.
- **User Interfaces:** Ensuring responsiveness by handling computations and I/O concurrently.
- **Latency Hiding:** In network- and disk-bound programs, concurrency hides the latency associated with I/O operations.

Example – A Word Processor

A typical word processor must simultaneously:

- *Read keyboard input.*
- *Update and refresh the display.*
- *Format text and update internal data structures.*
- *Periodically autosave documents.*
- *Perform spell-checking.*
- *Possibly execute other background tasks.*

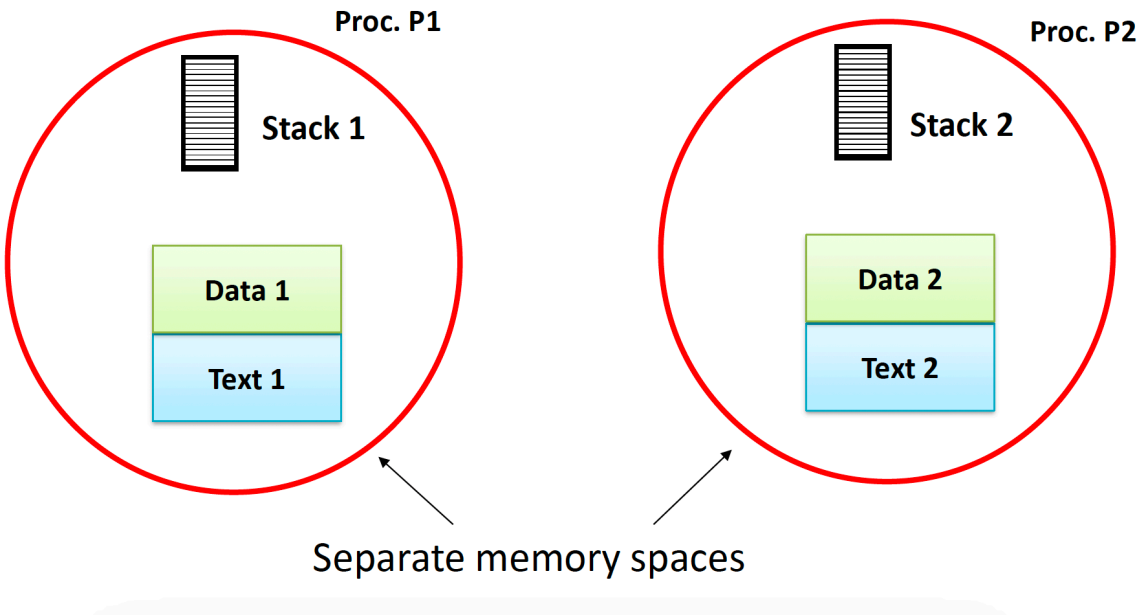
Concurrency by Process Decomposition

- One idea is to implement each function (e.g., input handling, screen update, autosave) in a separate process.
- **Issue:** Processes have separate private memory spaces. Communication between them must be achieved through inter-process communication (IPC) mechanisms such as pipes or socket-based message passing.
- **Trade-Off:** While IPC allows separation of concerns, it usually incurs higher communication overhead than shared-variable approaches.

Inter-Process Communication (IPC) vs. Threads

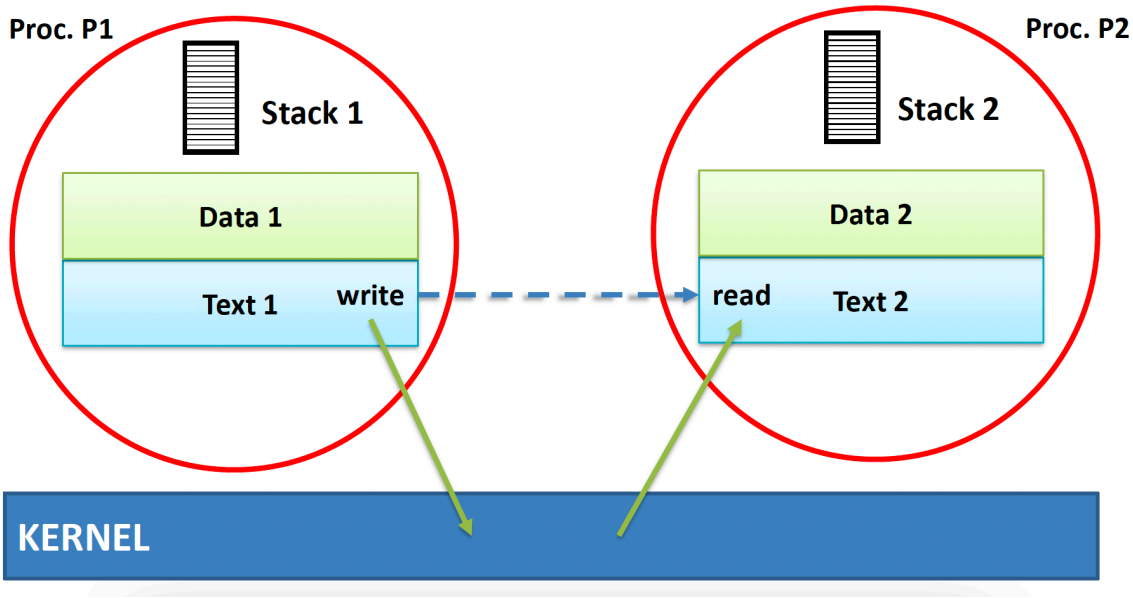
IPC Overview

- **Separate Memory Spaces:** By design, separate processes do not share memory, which enforces protection but complicates direct sharing of state.



- **Mechanisms:**
  - Pipes (named or unnamed).
  - Sockets (for send/receive operations).

- Other IPC techniques (e.g., message queues).



Threads as an Abstraction

- **Definition:** A thread is a single execution sequence that is independently schedulable. In other words, a thread represents one flow of control within a process.
- **Key Characteristics:**
  - **Single Sequential Flow:** Mirrors the traditional programming model.
  - **Independent Scheduling:** The OS or thread library can suspend, resume, or run a thread independently.
  - **Shared Environment:** Threads within the same process share the same address space and resources, allowing efficient sharing of data.
- **Why Use Threads?**
  - *Simplifies program structure.*
  - *Improves responsiveness (for example, in UI applications).*
  - *Exploits multi-core hardware.*
  - *Allows interaction with slow I/O devices without stalling the entire process.*

- **Protection Consideration:** Protection is separate from threading; a process may have one or many threads within a single protection domain.

Thread Models and Implementations

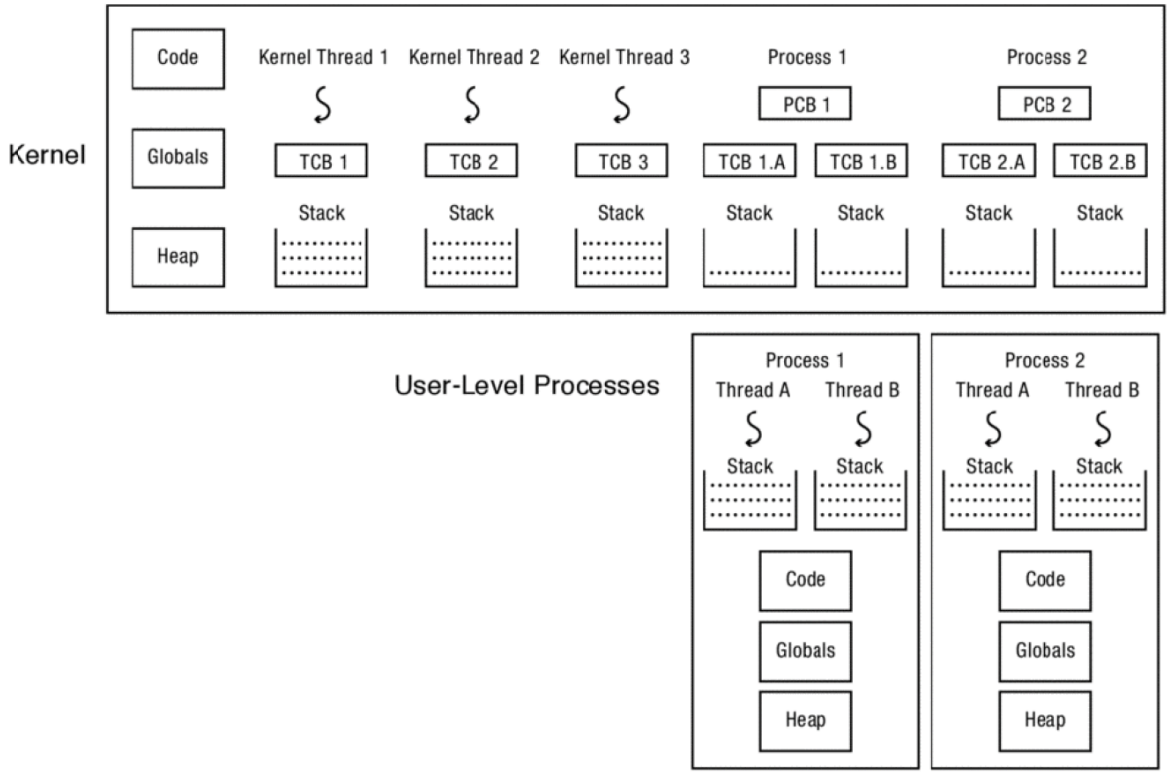
Types of Thread Implementations

1. Multiprocess Kernel with Multiple Single-Threaded Processes

- Each process is independent and uses system calls to access shared kernel data structures.

2. Multi-Threaded Kernel

- The kernel supports multiple threads within the same process. Threads share kernel data structures and can execute privileged instructions.
- This model allows different threads in the same process to run in parallel on multiple processors/cores.



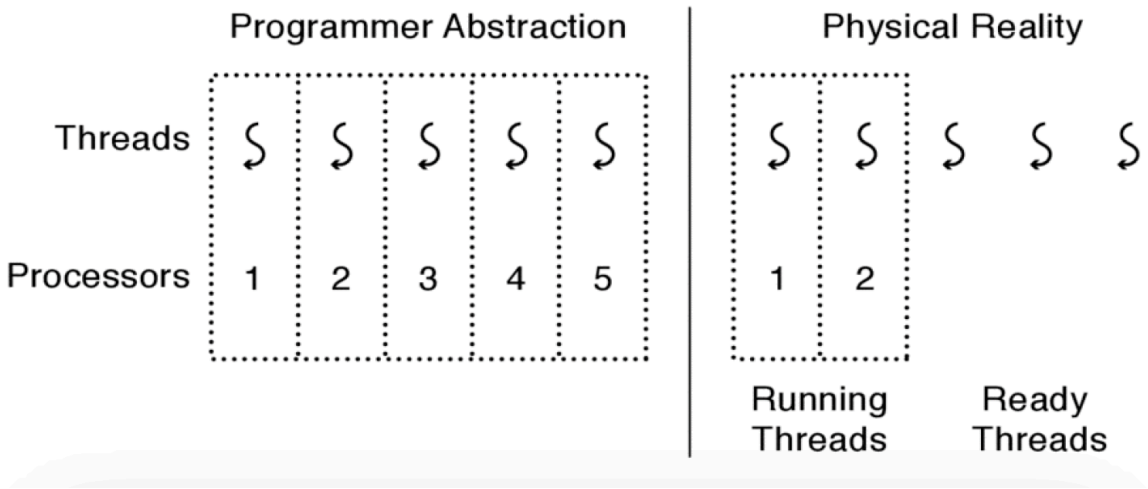
3. Single-Threaded User Programs

- The process has only one thread, making the process equivalent to a thread.

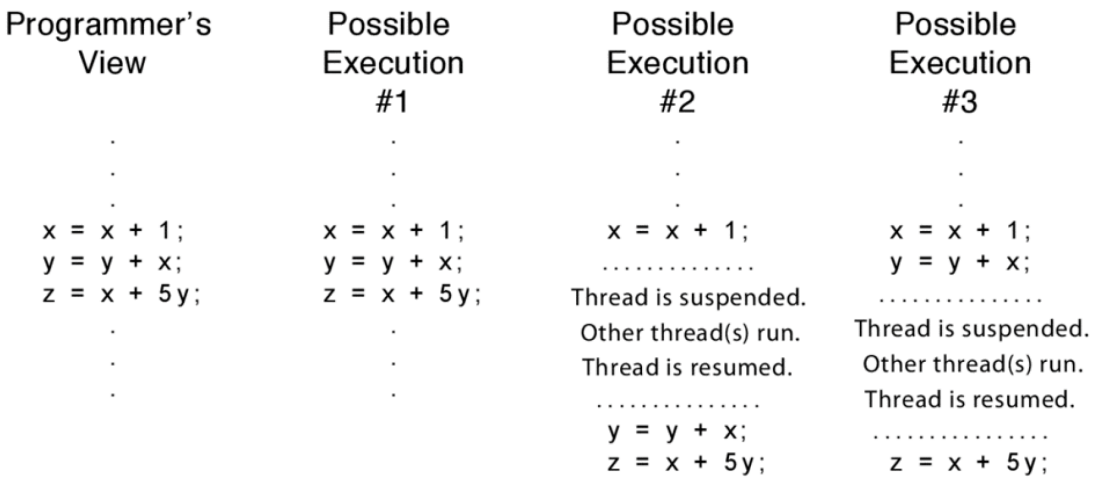
4. Multi-Threaded User Programs

- Multiple threads exist within the user process; they share data structures but are isolated from other processes.

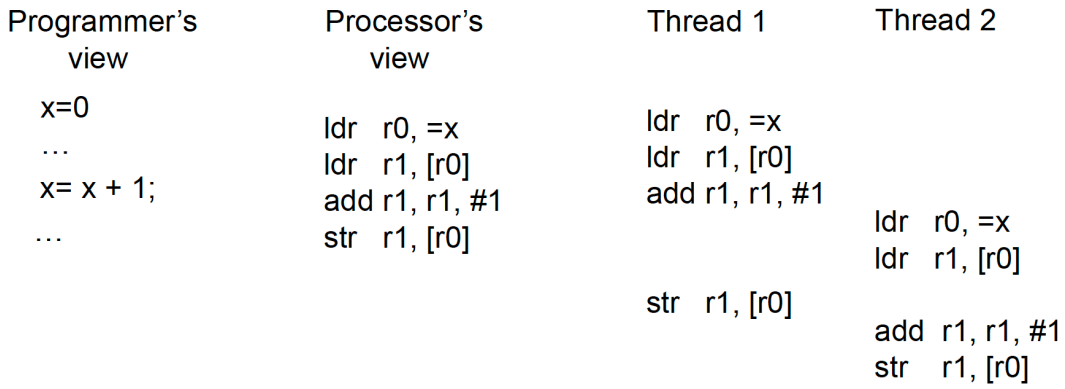
Thread Abstraction Details



- **Unlimited Processors View:** Conceptually, one can imagine a dedicated processor for each thread.
- **Variable Execution Speed:** Threads may execute at different speeds, and the program must be designed to function correctly under any scheduling order.
- **Programmer vs. Processor View:**
  - At the programmer level, statements such as `x = x + 1;` are written as atomic high-level operations.



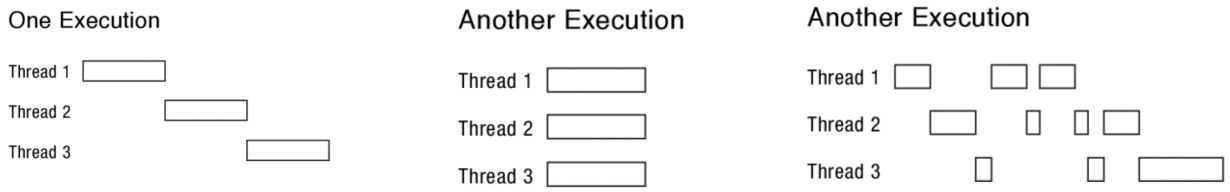
- At the processor level, the same statement is implemented through a sequence of instructions (e.g., load, add, store), where an interrupt can occur between instructions—hence, the operation is not atomic by default.



Concurrency Challenges Illustrated by Instruction-Level Interleaving

- When two threads execute the same sequence concurrently on a shared variable (e.g., incrementing a variable `x`), an interrupt can occur between the load and store instructions. This interleaving may result in incorrect final values for `x`, demonstrating the need for synchronization mechanisms when multiple threads access shared data.

Possible execution of threads



Thread Management and Lifecycle

Thread Control Block (TCB)

- **Purpose**: Stores all information needed to manage and schedule a thread.
- **Contents May Include**:
  - *Thread identifier.*
  - *Current state* (`INIT`, `READY`, `RUNNING`, `WAITING`, `FINISHED`).
  - *Processor register state.*
  - *Scheduling parameters (**priority**, **affinity**).*
  - *Pointer to the thread's stack and thread-local storage.*

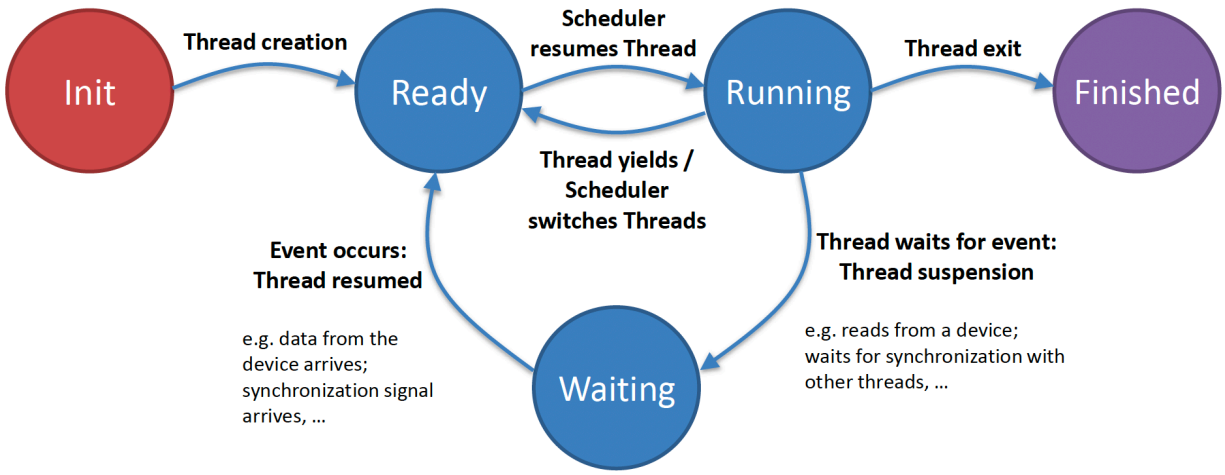
Basic Thread Operations (API)

- `thread_create(th, func, args)`
  - Allocates a new TCB.
  - Allocates a stack for the new thread.
  - Constructs the initial stack frame (often via a stub that calls the provided function).
  - Places the thread in the ready list so that it will run in due course.
- `thread_yield()`
  - The calling thread voluntarily relinquishes the processor, allowing the scheduler to run another thread.
- `thread_join(th)`
  - Blocks the calling thread until the specified thread `th` completes.

    - Can only be called once per thread to collect its exit status.
- `thread_exit(exitstatus)`
    - Terminates the current thread, recording the exit status in its TCB.
    - Moves the thread to a finished state, after which its resources can be reclaimed by another thread.

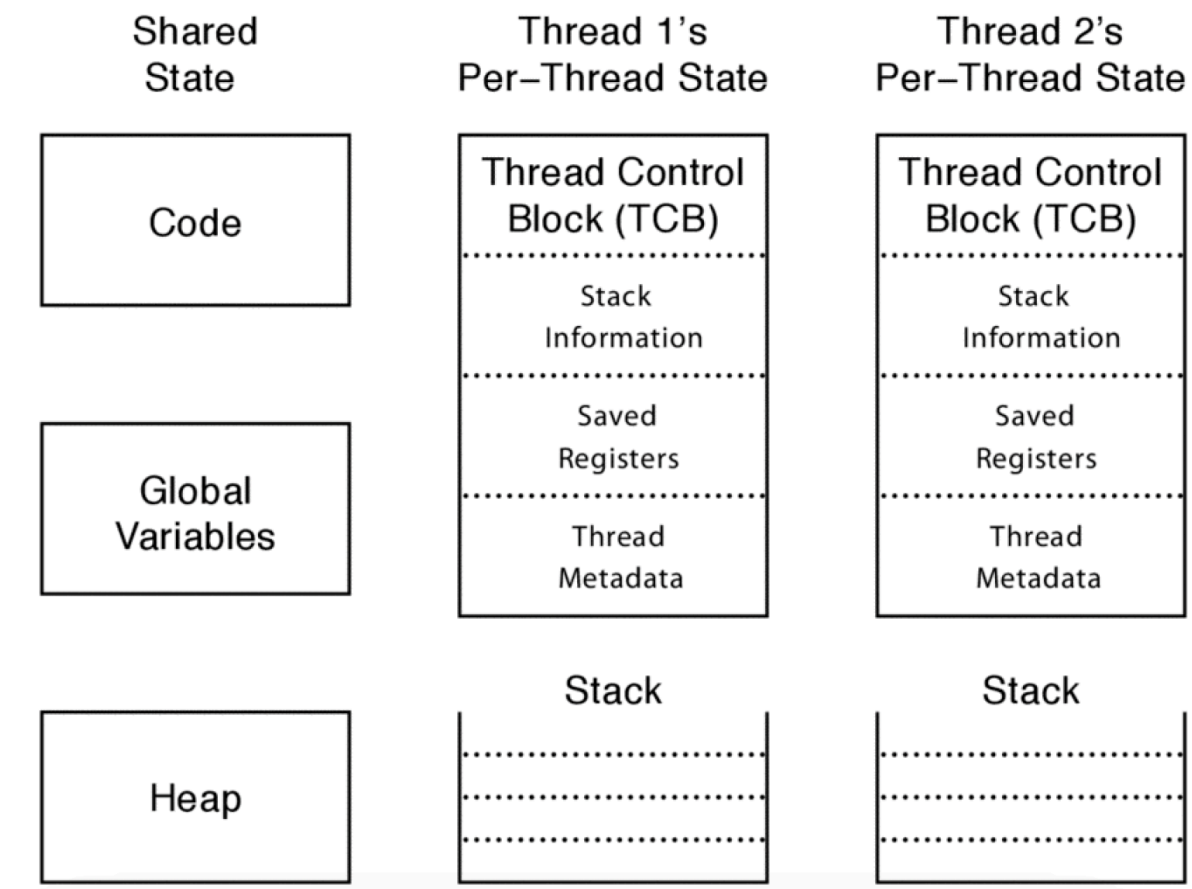
Thread Lifecycle States

- `INIT`: Thread is being created.
- `READY`: Thread is prepared and waiting in the ready list.
- `RUNNING`: Thread is currently executing on a processor.
- `WAITING`: Thread is blocked, waiting for an event (e.g., I/O completion, synchronization signal).
- `FINISHED`: Thread has completed execution and awaits cleanup.



Shared vs. Per-Thread State

- **Shared State**: Variables and resources common to all threads within a process.
- **Per-Thread State**: Registers, stack pointers, and other context specific to each thread, stored in the TCB.

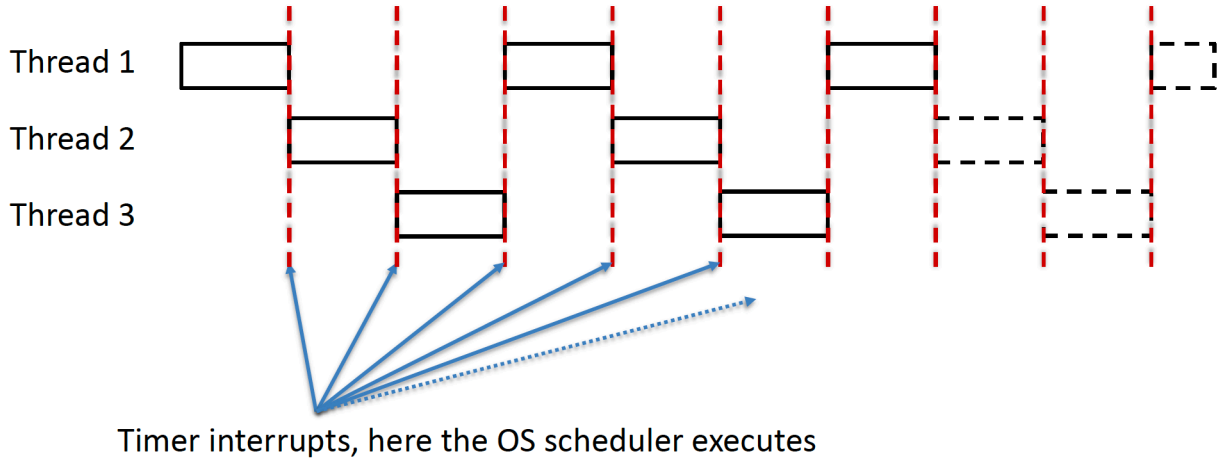


State of thread	Location of TCB	Location of registers
INIT	Being created	TCB
READY	Ready List	TCB
RUNNING	Running List	Processor
WAITING	Synchronization Variable's Waiting List	TCB
FINISHED	Finished List, then Deleted	TCB

Scheduling and Context Switching

Scheduler in Time-Sharing Systems

- **Role**: The scheduler determines which thread runs at any given time, based on a scheduling algorithm.
- **Context Switch Triggers**:
  - **Voluntary**: When a thread calls thread_yield() or blocks on a synchronization primitive.
  - **Involuntary**: Triggered by timer interrupts or preemptive scheduling decisions by the OS.



Cooperative vs. Preemptive Multi-Threading

- **Cooperative Multi-Threading**:
  - Threads must explicitly yield control.
  - Provides predictable interleavings but risks a long-running thread monopolizing the CPU.
  - Less common in modern systems.
- **Preemptive Multi-Threading**:
  - The scheduler can interrupt a running thread at almost any time.
  - Better suited for modern systems with multi-core processors and for maintaining responsiveness.

- Generally, context switches are more costly compared to cooperative threading.

Thread Context Switch Mechanism

What Happens During a Context Switch:

1. **Saving the Old Thread's Context:**
   - The processor saves the current thread's register values (including `PC`, `PS`, `SP`, general registers) into its `TCB`.
2. **Updating the Scheduler:**
   - The old thread is moved to the appropriate list (ready or waiting).
   - The scheduler selects a new thread from the ready list.
3. **Restoring the New Thread's Context:**
   - The new thread's context is loaded from its `TCB`.
   - The processor resumes execution in the new thread's context.

Example of Context Switching (Software and Hardware View)

- **Hardware Support:** The processor might use special instructions (e.g., IRET on x86) to return from an interrupt and resume execution.
- **Kernel vs. User-Level Differences:**
  - In user-level threads, the thread library saves and restores context on the user stack.
  - In kernel-level threads, system calls and interrupts trigger context switches where the kernel's scheduler takes over.

Overhead Considerations:

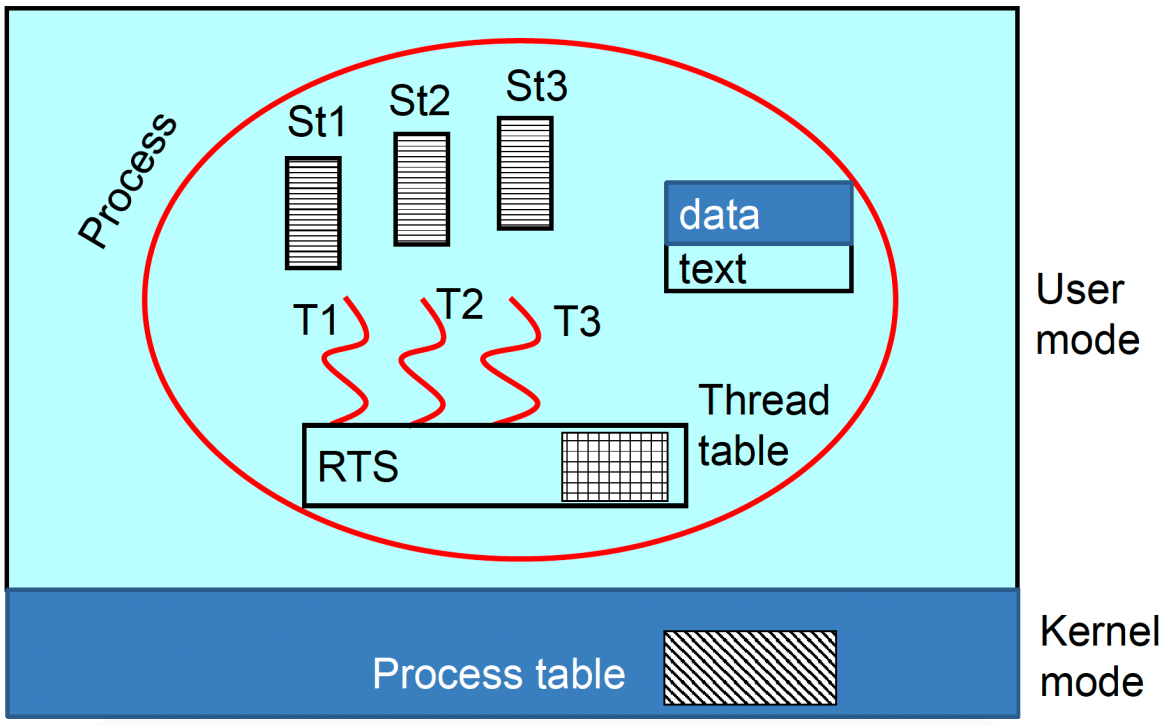
- **Register Save/Restore:** The time to save and load registers.
- **TCB Queue Management:** Overhead in managing the ready, waiting, and finished lists.

- **Cache Effects:** Cache invalidation might occur during a switch, especially if switching between threads from different processes.
- **Memory Manager Involvement:** Potential page faults or MMU/TLB invalidation if threads come from different processes or if the memory mappings change.

Thread Implementation Details

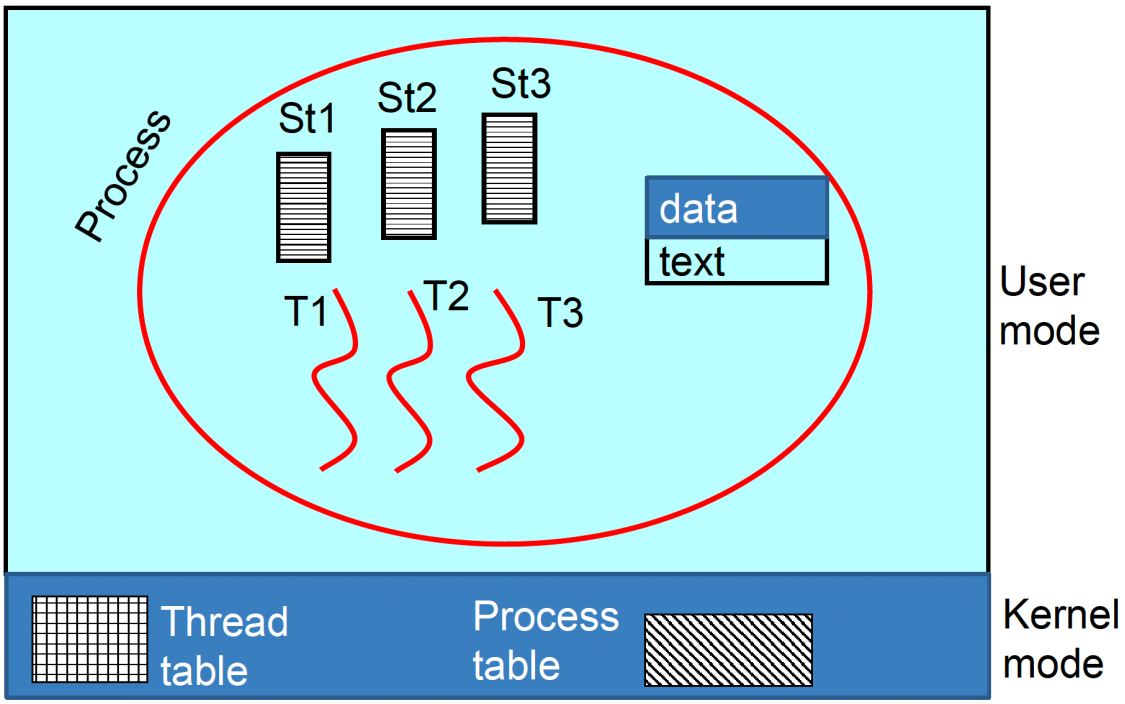
User-Level Threads

- **Implementation:** Managed entirely by a user-level library.
- **Characteristics:**
  - Cooperative scheduling (unless enhanced with mechanisms like Scheduler Activations).
  - Efficient context switches (no kernel mode transitions).
  - **Limitations:** A blocking system call in one thread can block all threads in the process; typically, these threads do not fully exploit multi-core architectures unless combined with scheduler activation techniques.
- **Typical Structure:** A thread table maintained in user space, with the runtime system performing scheduling and context switching.



Kernel-Level Threads

- **Implementation**: Managed directly by the operating system kernel.
- **Characteristics**:
  - System calls are used for creation, termination, and context switching.
  - Preemptive scheduling with support for multiple processors.
  - **Trade-Off**: Higher overhead due to kernel transitions and more complex context-switching routines.
- **Kernel Data Structures**:
  - The kernel maintains a global thread table.
  - Each process may have a thread table for its user-level threads, while kernel threads are scheduled system-wide.



Hybrid Approaches

- **Scheduler Activations**:
  - A technique that allows user-level thread libraries to coexist with kernel-level scheduling.
  - The kernel assigns processors to the user-level scheduler, which then manages context switches.
  - This model helps mitigate the blocking problems of pure user-level threads.
- **Asynchronous I/O Combined with Threads**:
  - Modern systems often use asynchronous I/O to minimize blocking while still leveraging multi-threading for parallelism.

Process Control Block (PCB)

The **PCB** is a data structure used by the operating system to manage a **process**. It contains all the necessary information to execute and control a process. Some typical fields in a PCB include:

- **Process ID (PID)**: A unique identifier for the process.
- **Process state**: Ready, running, blocked, terminated, etc.
- **CPU context**: Values of registers, program counter, stack pointer, etc.
- **Memory management information**: Code, data, heap, and stack segments.
- **File management information**: Open files, file descriptors.
- **Resource management information**: CPU usage, execution time, priority.
- **Thread list**: If the process is multithreaded, the PCB tracks all associated threads.

A **process has only one PCB**, even if it has multiple threads.

Thread Control Block (TCB)

The **TCB** is a data structure used by the operating system to manage an individual **thread** within a process. Since a process can have multiple threads, there will be **one TCB per thread**. Some typical fields in a TCB include:

- **Thread ID (TID)**: A unique identifier for the thread.
- **Thread state**: Ready, running, blocked, terminated, etc.
- **CPU registers**: Each thread has its own execution context (program counter, stack pointer, etc.).
- **Thread stack**: Each thread has its own separate stack.
- **Thread priority**: Priority relative to other threads.
- **Pointer to the parent process's PCB**: To maintain the relationship between the thread and its process.

Key Differences Between PCB and TCB

Feature	PCB (Process Control Block)	TCB (Thread Control Block)
Purpose	Stores information about a **process**	Stores information about a **thread**

Unit managed	Process (which may contain multiple threads)	Individual thread
Associated ID	PID (Process ID)	TID (Thread ID)
State tracking	Process state	Thread state
CPU context	Global process context	Thread-specific context
Memory management	Includes entire process memory (heap, global stack, code)	Each thread has its own stack but shares global process memory
Stack	A process may have multiple stacks (one per thread)	Each thread has its own separate stack

Relationship Between PCB and TCB

- Each **process** has **one PCB**.
- Each **thread** has **its own TCB**.
- The **PCB contains references to the TCBs** of all threads belonging to that process.